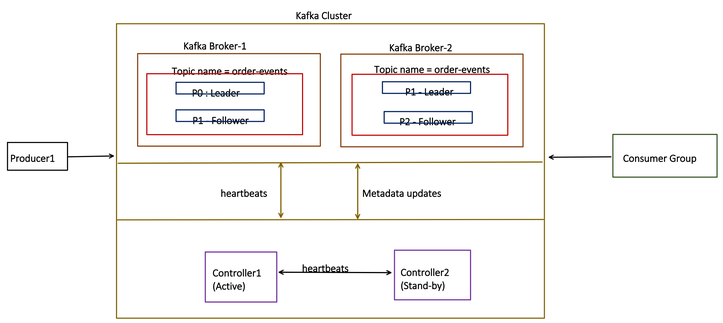
Need for Kafka Stream (KStream)

Wednesday, 3 June 2026 12:21 PM

First lets understand the need for Kafka Stream:

So till now we have established very good understanding of:

- Kafka Producer
- Kafka Cluster
- Kafka Consumer and Consumer Groups



So current flow is:

Producer publish events
→ Events stored in particular Topic Partition
→ Consumer read the events from the partition
→ Process them (like send notification)

From above flow, we can easily notice few things:

- In Kafka Consumer, each event is processed independently, means no memory of previous events/messages needed.
- Output depends on the current event only.
- And there is no need to remember anything from previous events or messages.

So this is known as STATELESS processing.

But

What if we need STATEFUL processing?

Means:

- Output depends on the previous events /messages too.
- Must remember (or maintain states) across messages.

This will break Kafka Consumer (as Consumer API are designed for Stateless processing).

Real World Problems:

---

1. Real time Aggregation:
   Use-cases:
   - Real time total revenue per customer.
     Ex:
     Customer1: 100$

Customer2: 150$

- Real time Leaderboard.
  EX:
  UserC: 50 points
  UserA: 35 points
  UserB: 15 points

Etc.

with plain Kafka Consumer, we might be doing:

Approach-1: Storing state in-memory

```
@Component
public class RevenueConsumer {

  private final Map<String, Double> revenueMap = new ConcurrentHashMap<>();

  @KafkaListener(topics = "order-events")
  public void consume(OrderEvent event) {

    revenueMap.merge(
        event.getCustomerId(),
        event.getAmount(),
        Double::sum);
  }
}
```

Problem:

1. Application Crash:
   - In memory Map gone.
   - We need to Relay from offset 0 (millions of messages)

2. Multiple Instances:
   - Each instance maintain PARTIAL data as each instance reads different partition.

Approach-2 Storing State in MySQL:

```
@KafkaListener(topics = "order-events")
public void consume(OrderEvent event) {

  CustomerRevenue revenue = repository.findById(event.getCustomerId());

  revenue.setAmount(revenue.getAmount() + event.getAmount());

  repository.save(revenue);
}
```

Problem:

For every Kafka Event ---> SELECT ---> UPDATE.
Assume if we have 1,00,000 events / sections then we need to do:

1,00,000 - SELECT QUERY per second
1,00,000 - UPDATE QUERY per second

Our database will become bottleneck.

And this is just one Problem "Real time Aggregation" but what if more requirements are added like:

- Window operations
  User-Case:
  - Total revenue in LAST 5 MINUTES

Problems With plain Kafka Consumer:
#1. We need to track timestamp manually.
#2. Expire old entries manually.
#3. Multiple instance, each sees partial window data.

- Joining of 2 Topics
  User-Case:

- *"order-events"* has *customer_id*, so join it with *"customer-profile"* to fetch Customer name.

Problems With plain Kafka Consumer:

#1: We need to maintain the map of Consumer profiles

#2: What if this in-memory Map lost because of App crash, need to be rebuild by replaying from Offset 0

#3. If multiple Consumer instance:

Consumer-A: maintains P0 of both the topics *"order-events"* and *"customer-profile"*

"order-events-P0"

"consumer-profile-P0"

Consumer-B: maintains P1 of both the topics *"order-events"* and *"customer-profile"*

"order-events-P1"

"consumer-profile-P1"

Now what if Consumer-A when order-event arrives, want to read the consumer entry which is maintained by Consumer B? So we need Distributed State Sharing.

So, if we try to manually solve these problem, we might end bud building STREAM framework itself which has:

- State management
- Windowing
- Joins
- Crash recovery (rebuild state)
- Scaling (Partitions divide among multiple worker nodes)

Etc.